

RoboPet - Combat Overhaul

Abilities, Overcharge, Loadout, Types, Loot and Bosses

Cursor task - build in phases, paste one phase at a time

This turns the passive auto-battle into an active, strategic system. Build in this order; each phase is playable on its own.

Prerequisite: apply the level-resume fix first (persist combat_level so progression sticks).

1. Phase A - Active abilities (Strike / Shield / Repair + cooldowns)
2. Phase B - Overcharge meter + timed special move
3. Phase C - Status effects engine (overheat / stun / shield / EMP)
4. Phase D - Enemy types + weaknesses (uses Phase C)
5. Phase E - Module loadout drives combat (ties to Fabricator)
6. Phase F - Loot drops + boss every 5 levels

After each phase: flutter pub get, hot restart, verify that phase in the Acceptance Test.

Acceptance test (by phase)

Phase A

- ☐ Three ability buttons appear during a fight.
- ☐ Strike deals a clear burst, Shield blocks the next enemy hit, Repair heals.
- ☐ Each ability shows a cooldown and cannot be spammed.

Phase B

- ☐ A charge meter fills as you fight, and lights up when full.
- ☐ Tapping the special during the sweet spot deals max damage, off-spot deals less.
- ☐ Using the special resets the meter.

Phase C

- ☐ Overheat ticks damage over time, Stun makes a side skip a turn, Shield absorbs damage, EMP stops regen.
- ☐ Status icons and timers are visible and count down.

Phase D

- ☐ Each level has an enemy type, and the UI shows what beats it.
- ☐ Hitting a weakness deals noticeably more damage.

Phase E

- ☐ Equipping different modules in the Fabricator changes combat stats and ability power.
- ☐ The pre-fight screen shows a loadout summary.

Phase F

- [] Winning sometimes drops a module part or blueprint, added to inventory.
- [] Levels 5 / 10 / 15 are bosses with a telegraphed heavy attack you must block or dodge.

Phase A - Active abilities

Goal: hybrid combat - the auto-tick keeps running, but the player taps abilities with cooldowns.

```
enum Ability { strike, shield, repair }

const kAbilityCooldown = <Ability, double>{
    Ability.strike: 3.0,
    Ability.shield: 6.0,
    Ability.repair: 8.0,
};

// Add to combat state:
// Map<Ability, double> cooldowns (seconds remaining, 0 = ready)
// int shieldHp (absorbs incoming damage)

void useAbility(Ability a) {
    if ((state.cooldowns[a] ?? 0) > 0) return; // still cooling down
    switch (a) {
        case Ability.strike:
            final dmg =
                ((state.playerAtk * 1.8) - state.enemyDef * 0.3).round().clamp(1, 9999);
            _damageEnemy(dmg);
            break;
        case Ability.shield:
            state = state.copyWith(shieldHp: state.shieldHp + 25);
            break;
        case Ability.repair:
            final hp = (state.playerHp + 30).clamp(0, state.playerMaxHp);
            state = state.copyWith(playerHp: hp);
            break;
    }
    final cd = Map<Ability, double>.from(state.cooldowns)..[a] = kAbilityCooldown[a!];
    state = state.copyWith(cooldowns: cd);
}

// Decrement cooldowns every frame or tick.
void tickCooldowns(double dt) {
    final cd = Map<Ability, double>.from(state.cooldowns);
    for (final a in cd.keys.toList()) {
        cd[a] = (cd[a]! - dt).clamp(0, 999);
    }
    state = state.copyWith(cooldowns: cd);
}
```

Route enemy damage through the shield first:

```
void _applyEnemyHit(int raw) {
    var dmg = raw;
    if (state.shieldHp > 0) {
        final absorbed = dmg.clamp(0, state.shieldHp);
        dmg -= absorbed;
        state = state.copyWith(shieldHp: state.shieldHp - absorbed);
    }
    final hp = (state.playerHp - dmg + state.playerRegen).clamp(0, state.playerMaxHp);
}
```

```

    state = state.copyWith(playerHp: hp);
}

```

Phase B - Overcharge meter + timed special

Goal: build a charge bar, then release a big special whose power depends on tap timing.

```

// Add to state: double charge (0..100). specialReady => charge >= 100.

void _addCharge(double amount) {
    state = state.copyWith(charge: (state.charge + amount).clamp(0, 100));
}

// timingQuality is 0..1 (1 = tapped dead center of the sweet spot).
void useOvercharge(double timingQuality) {
    if (state.charge < 100) return;
    final mult = 2.0 + 1.5 * timingQuality; // 2.0x .. 3.5x
    final dmg = ((state.playerAtk * mult) - state.enemyDef * 0.3).round().clamp(1, 9999);
    _damageEnemy(dmg);
    _applyStatus(toEnemy: true, kind: StatusKind.overheat, ticks: 3, magnitude: 6);
    state = state.copyWith(charge: 0);
}

```

UI (yours): a meter that fills on each landed hit (call `_addCharge`), plus a moving needle when full - `timingQuality = 1 - distanceFromCenter`.

Phase C - Status effects engine

Goal: a small engine for overhear (DoT), stun (skip turn), shield (absorb), EMP (block regen).

```

enum StatusKind { overheat, stun, shield, emp }

class StatusEffect {
    final StatusKind kind;
    int ticksLeft;
    final int magnitude;
    StatusEffect(this.kind, this.ticksLeft, this.magnitude);
}

// State: List<StatusEffect> playerStatuses, enemyStatuses.

// Returns damage-over-time to apply this tick, and ages the effects.
int resolveStatuses(List<StatusEffect> list) {
    var dot = 0;
    for (final s in list) {
        if (s.kind == StatusKind.overheat) dot += s.magnitude;
        s.ticksLeft -= 1;
    }
    list.removeWhere((s) => s.ticksLeft <= 0);
    return dot;
}

bool isStunned(List<StatusEffect> l) => l.any((s) => s.kind == StatusKind.stun);
bool regenBlocked(List<StatusEffect> l) => l.any((s) => s.kind == StatusKind.emp);

```

In `_tick`: apply DoT to each side, skip a side attack if `isStunned`, and add regen only if not `regenBlocked`.

Phase D - Enemy types + weaknesses

Goal: rock-paper-scissors damage types so picking the right core matters.

```
enum DamageType { kinetic, emp, fire, acid }

// Attacker type beats this defender type for +50% damage.
const kBeats = <DamageType, DamageType>{
    DamageType.kinetic: DamageType.acid,
    DamageType.acid: DamageType.fire,
    DamageType.fire: DamageType.emp,
    DamageType.emp: DamageType.kinetic,
};

double weaknessMultiplier(DamageType attacker, DamageType defender) =>
    kBeats[attacker] == defender ? 1.5 : 1.0;

DamageType enemyTypeForLevel(int level) =>
    DamageType.values[(level - 1) % DamageType.values.length];
```

Multiply player damage by `weaknessMultiplier(playerType, enemyType)`. Show the enemy type plus what beats it (the sensor module can reveal it).

Phase E - Module loadout drives combat

Goal: the modules crafted in the Fabricator become the combat build. This closes the economy loop.

```
class CombatLoadout {
    final int bonusAtk;      // core
    final int bonusDef;      // armor
    final int bonusMaxHp;    // armor
    final int bonusSpd;      // wheels
    final double critChance; // sensor
    final double cooldownScale; // utility (<1 = faster)
    final double chargeRate; // core
    final DamageType type;   // core element
    const CombatLoadout({
        this.bonusAtk = 0,
        this.bonusDef = 0,
        this.bonusMaxHp = 0,
        this.bonusSpd = 0,
        this.critChance = 0,
        this.cooldownScale = 1.0,
        this.chargeRate = 10,
        this.type = DamageType.kinetic,
    });
    // Add a copyWith (or use your state class pattern).
}

CombatLoadout loadoutFromModules(List<Module> equipped) {
    var l = const CombatLoadout();
    for (final m in equipped) {
        switch (m.slot) {
            case ModuleSlot.core:
                l = l.copyWith(
                    bonusAtk: l.bonusAtk + m.power,
                    chargeRate: l.chargeRate + m.power.toDouble(),
                    type: m.element,
                );
        }
    }
}
```

```

        break;
    case ModuleSlot.armor:
        l = l.copyWith(bonusDef: l.bonusDef + m.power, bonusMaxHp: l.bonusMaxHp +
m.power * 3);
        break;
    case ModuleSlot.wheels:
        l = l.copyWith(bonusSpd: l.bonusSpd + m.power);
        break;
    case ModuleSlot.sensor:
        l = l.copyWith(critChance: l.critChance + m.power * 0.01);
        break;
    case ModuleSlot.utility:
        l = l.copyWith(cooldownScale: (l.cooldownScale - m.power * 0.02).clamp(0.4,
1.0));
        break;
    }
}
return l;
}

```

Apply the loadout in `startFight()`: add the bonuses to base stats, set the player damage type, and multiply ability cooldowns by `cooldownScale`. Show a loadout summary on the pre-fight screen.

Phase F - Loot drops + bosses

Goal: a reason to keep fighting (loot feeds the Fabricator) and a skill check every 5 levels.

```

import 'dart:math';

bool isBossLevel(int level) => level % 5 == 0;

class LootDrop {
  final int scrap;
  final Object? part; // a module part or blueprint, or null
  const LootDrop({required this.scrap, this.part});
}

LootDrop rollLoot(int level, bool boss) {
  final scrap = (10 + level * 5) * (boss ? 3 : 1);
  final partChance = boss ? 1.0 : (0.25 + level * 0.01);
  final part = (Random().nextDouble() < partChance)
    ? randomModulePart(level: level, boss: boss)
    : null;
  return LootDrop(scrap: scrap, part: part);
}

```

Boss telegraphed heavy attack - the player must raise Shield (or dodge) in time:

```

void startBossWindup() {
  state = state.copyWith(bossWindup: true); // show a wind-up indicator
  Future.delayed(const Duration(milliseconds: 1500), _resolveBossHit);
}

void _resolveBossHit() {
  final blocked = state.shieldHp > 0; // raised Shield in time
  final raw = (state.enemyAtk * (blocked ? 0.4 : 2.5)).round();
  _applyEnemyHit(raw);
  state = state.copyWith(bossWindup: false);
}

```

On victory, call rollLoot, add scrap and any part to inventory, then unlock the next level. On boss levels, trigger startBossWindup once or twice during the fight.

Notes

- Adapt names: Module, ModuleSlot, m.power, m.element, randomModulePart, inventory add, CombatPhase, and state field names will differ in your code. The structure and logic are what matter.
- copyWith: add copyWith to your combat state and to CombatLoadout, or fold them into your existing state class.
- UI is yours: ability buttons, the charge bar with a moving needle, status icons, the type / weakness badge, the loadout summary, the loot popup, and the boss wind-up indicator.
- Balance: tune all numbers after playing; layer this on top of enemyForLevel from the previous guide rather than replacing it.
- Build incrementally: ship Phase A first, play it, then add the next phase. Do not try to land all six at once.